

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I Samiksha Dhanya G (192321014) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Samiksha Dhanya G (192321014)

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

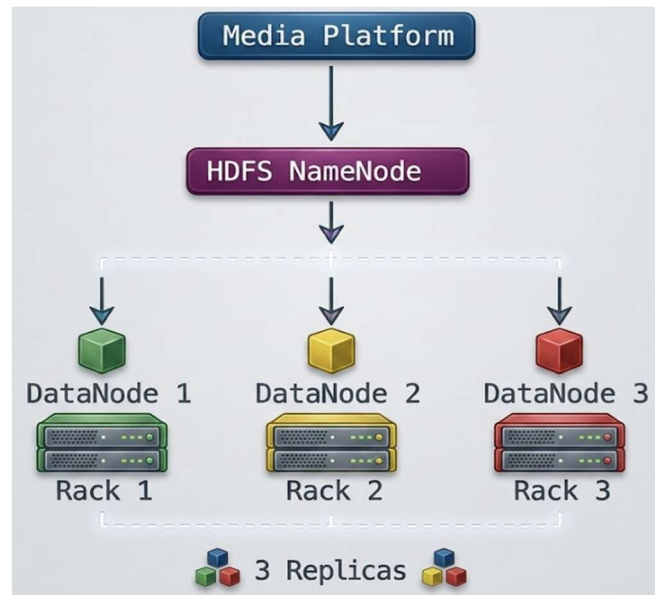
Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood

System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Solution:

1. HDFS-Based Storage Solution for a Media Platform

Architecture Diagram



Components

- **NameNode:** Maintains HDFS metadata and file locations.
- **DataNodes:** Store media blocks.
- **Replication factor = 3:** Maintains three copies of important data.
- **Rack awareness:** Places replicas across different racks.
- **Large block size:** Suitable for large video and image files.

Working

1. Media files are divided into HDFS blocks.
2. Blocks are stored across multiple DataNodes.
3. Each block is replicated three times.
4. HDFS uses rack-aware placement so replicas are not concentrated in one rack.
5. New DataNodes can be added when storage requirements increase.
6. Frequently accessed media can use replication, while cold data can use erasure coding where appropriate.

Constraint Satisfaction

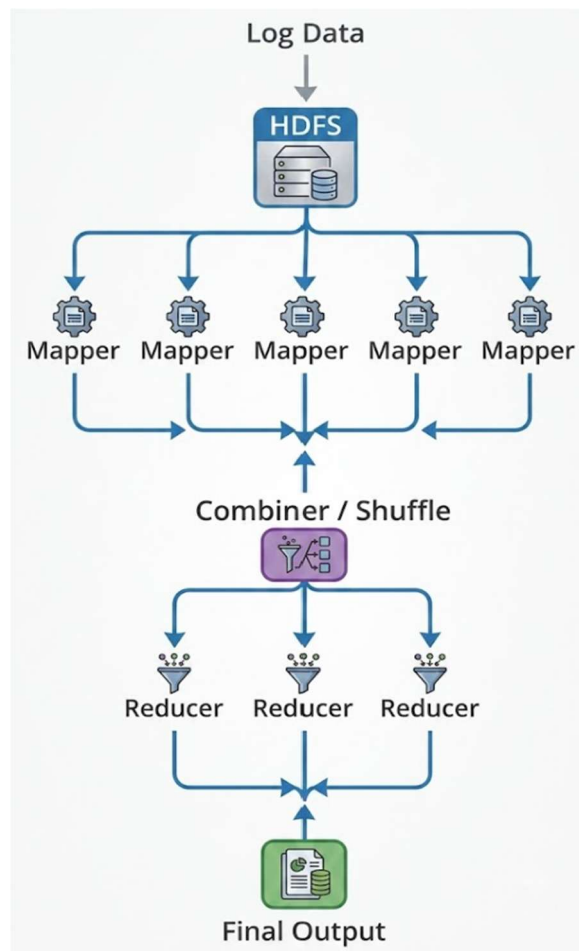
Constraint	Solution
Availability > 99.9%	3 replicas + rack awareness
Replication ≤ 3	Replication factor fixed at 3
Cost efficient	Horizontal scaling + large blocks + erasure coding for cold data

Justification

The design provides high availability because failure of a DataNode or rack does not make the media unavailable. The replication factor of three satisfies the maximum replication constraint. Horizontal scaling allows storage capacity to grow gradually without excessive initial investment.

2. MapReduce Workflow for Log Processing

Architecture Diagram



Components

- **HDFS:** Stores log files.
- **Mapper:** Processes individual log blocks.
- **Combiner:** Performs local aggregation.
- **Shuffle and Sort:** Transfers mapper output to appropriate reducers.
- **Reducer:** Produces final aggregated results.

Working

1. Log files are divided into HDFS blocks.
2. Mapper tasks are assigned to nodes containing the required blocks.
3. Multiple mappers execute in parallel.
4. A combiner reduces intermediate data locally.
5. Shuffle and sort distributes data to reducers.
6. Reducers generate the final output.

For example:

Log:

192.168.1.1 /home

192.168.1.2 /home

192.168.1.3 /login

Constraint Satisfaction

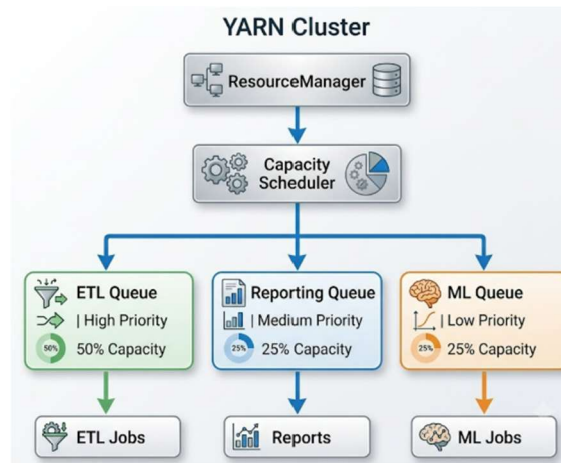
Constraint	Solution
20-minute batch window	Parallel mapper/reducer execution
Limited nodes	Efficient task distribution
Large log data	HDFS block-based processing
Network overhead	Combiner + compression
Slow task	Speculative execution if required

Justification

Data locality reduces network transfer because mapper tasks process data close to where it is stored. Parallel execution reduces processing time, while combiners reduce shuffle traffic. The number of reducers is selected according to available cluster resources so the job can finish within the 20-minute window.

3. YARN Scheduler for ETL, Reporting and ML

Architecture Diagram



Components

- **ResourceManager:** Manages cluster resources.
- **NodeManagers:** Manage resources on individual nodes.
- **Capacity Scheduler:** Allocates resources between queues.
- **ETL Queue:** Highest priority.
- **Reporting Queue:** Medium priority.
- **ML Queue:** Lower priority.

Working

1. Jobs enter their respective YARN queues.
2. The Capacity Scheduler allocates resources according to queue capacity.
3. ETL receives higher priority because data ingestion and transformation are time-sensitive.
4. Reporting receives a guaranteed resource allocation.
5. ML jobs use their allocated capacity and unused cluster resources.
6. If a NodeManager fails, YARN reschedules failed containers on another healthy node.
7. ResourceManager HA can be used to avoid a single point of failure.

Constraint Satisfaction

Constraint	Solution
Fairness	Separate queues with guaranteed capacity

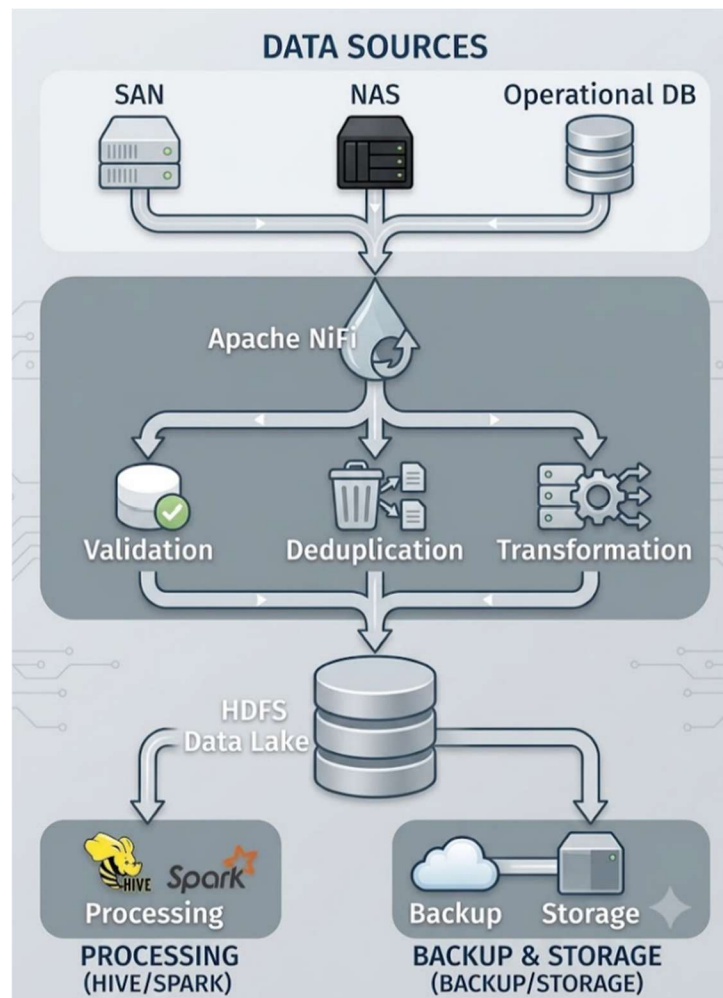
Constraint	Solution
ETL priority	Higher queue priority and capacity
Node failure	Container rescheduling
Resource efficiency	Unused capacity can be allocated to other queues

Justification

The Capacity Scheduler provides fairness by allocating resources to different queues while still allowing ETL jobs to receive higher priority. Node failures are handled by rescheduling containers on healthy nodes. ResourceManager high availability further improves cluster reliability.

4. Enterprise Data Ingestion from SAN, NAS and Databases

Architecture Diagram



Components

- **SAN:** Existing enterprise block storage.
- **NAS:** Existing file-based storage.
- **Operational databases:** Source of structured business data.
- **Apache NiFi:** Data ingestion and flow management.
- **HDFS:** Central big-data storage.
- **CDC/incremental extraction:** Captures only changed database records.
- **Hash/checksum:** Helps identify duplicate data.
- **Backup storage:** Provides disaster recovery.

Working

1. NiFi connects to SAN, NAS and operational databases.
2. Database data is extracted incrementally using CDC or timestamp-based extraction.
3. Files from SAN/NAS are transferred through NiFi.
4. NiFi validates and transforms incoming data.
5. Hashes or unique IDs are used to identify duplicate records.
6. Only new or changed data is stored in HDFS.
7. HDFS replication and separate backup storage provide data protection.

Constraint Satisfaction

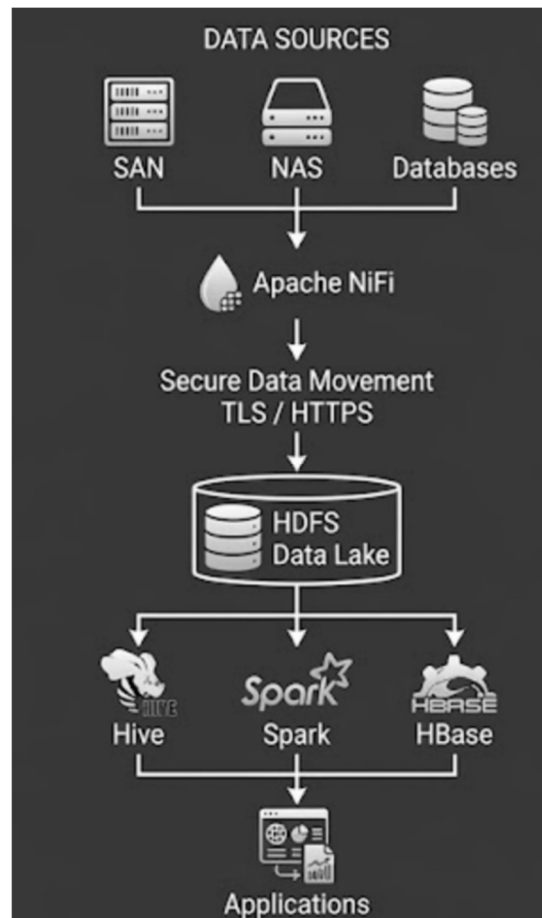
Constraint	Solution
Multiple data sources	NiFi connectors and ingestion flows
Backup reliability	HDFS replication + external backup
Minimal duplication	CDC + hashes/unique IDs
Large-scale storage	HDFS

Justification

Apache NiFi provides centralized data ingestion and monitoring for heterogeneous sources. Incremental extraction prevents repeatedly copying the same database data, while hashing and unique identifiers reduce duplication. HDFS replication and separate backup storage improve reliability and disaster recovery.

5. End-to-End Hadoop Ecosystem

Architecture Diagram



Components

- **Apache NiFi:** Data ingestion and ETL.
- **HDFS:** Distributed storage.
- **YARN:** Cluster resource management.
- **Hive:** SQL-based data analysis.
- **Spark:** Fast analytics and machine learning.
- **HBase:** Distributed NoSQL storage.
- **Kerberos:** Authentication.
- **TLS/SSL:** Secure data transfer.
- **HDFS replication:** Data availability.
- **NameNode HA:** Prevents NameNode single-point failure.
- **ResourceManager HA:** Provides YARN availability.

Working

1. Data is collected from databases, SAN and NAS.
2. NiFi manages the ingestion pipeline.
3. Data is transferred securely using TLS/HTTPS.
4. NiFi validates, transforms and routes the data.
5. Processed data is stored in HDFS.
6. HDFS replicates blocks across DataNodes.
7. Hive and Spark process the stored data.
8. YARN allocates cluster resources to applications.
9. Kerberos authenticates users and services.
10. NameNode and ResourceManager HA provide service continuity during failures.

Constraint Satisfaction

Constraint	Solution
Secure data movement	TLS/SSL + Kerberos
High availability	HDFS replication + NameNode HA + YARN HA
NiFi/ETL integration	NiFi ingestion and transformation pipelines
Scalable storage	HDFS
Data processing	Hive + Spark
Resource management	YARN

Justification

The proposed ecosystem provides secure end-to-end data movement through TLS/SSL and authenticated access using Kerberos. HDFS replication, NameNode HA and ResourceManager HA prevent individual component failures from causing major service disruption. Apache NiFi provides reliable ingestion and ETL integration, while Hive and Spark support large-scale analytics and processing.